

目次

PsiLang — Minimal ML-like Functional Language for Algorithms	1
構成	1
言語仕様 (概要)	1
動作確認	2
π 10,000 桁の計算	2
検証結果 (2026-05-16)	3
3 手法の比較 (遅 / 中 / 速)	3
v1 PhiLang との対比	4
限界	4

PsiLang — Minimal ML-like Functional Language for Algorithms

進化計算 (本ディレクトリ親 ../REPORT.md 参照) の top-5 個体から抽出したコンセンサス遺伝子を実装したもの。v1 の PhiLang が orchestration shell (compute = X の 1 行 dispatch) だったのに対し、PsiLang はアルゴリズムを言語自身の構文で書ける汎用 mini-ML である。

構成

ファイル	役割	行数
psilang.py	Lexer + Parser + AST + Tree-walker Evaluator + Stdlib (単一ファイル)	~570
test_basic.psi	動作確認: factorial / list / pattern match / Rat	15
chudnovsky.psi	π 10,000 桁を binary splitting で計算	50
pi_10k.txt	実行結果 (3 + 10,000 digits)	—
README.md	この文書	—

言語仕様 (概要)

- **paradigm:** functional (純粋関数ではない — 副作用は emit/print に限定)
- **type_system:** static_simple (型注釈は構文上受け取るが実行時は無視)
- **arithmetic_primitives:** Int (任意精度) + Rat (有理数、自動約分) + Real (任意精度小数、Decimal 由来)
- **control_flow:** if / match / general_recursion (no loops; iteration はパターンマッチ + 再帰)
- **syntax_family:** ml_like (let / fn / match / |)

文法 (要約)

fn name (p1: T1, p2: T2) -> Ret = body

```

let x = expr in body
let (a, b) = expr in body

if cond then a else b
match x with | pat1 -> e1 | pat2 -> e2 end

(a, b)          # 2-tuple
[1, 2, 3]       # list
x :: xs         # list cons, [] empty list

```

Builtin (一部)

- **Int:** + - * / mod ==, int_fact, int_pow, int_abs, int_sign, int_to_string
- **Rat:** rat(n, d), rat_add, rat_sub, rat_mul, rat_div, rat_neg, rat_inv, rat_num, rat_den, rat_from_int
- **Real:** real_from_int, real_from_rat(r, prec), real_sqrt(r, prec), real_mul, real_div, real_to_string(r, n), set_precision
- **Misc:** fst, snd, length, emit, print

動作確認

```

/usr/bin/python3 psilang.py test_basic.psi
# fact(20) = 2432902008176640000
# sum([1..5]) = 15
# 1+1/2+1/3+1/4 = 25/12
# int_fact(15) = 1307674368000
# int_pow(2, 10) = 1024

```

π 10,000 桁の計算

```

/usr/bin/python3 psilang.py chudnovsky.psi > pi_10k.txt
# 実時間: 3.03 秒 (Apple M2)

```

chudnovsky.psi の中身 (50 行) はアルゴリズムそのものを PsiLang で書いている:

```

fn term(k) =
  let sign = if k mod 2 == 0 then 1 else -1 in
  let num  = sign * int_fact(6 * k) * (545140134 * k + 13591409) in
  let den  = int_fact(3 * k) * int_pow(int_fact(k), 3) * int_pow(640320, 3 * k) in
  rat(num, den)

fn bsplit(a, b) =
  if b - a == 1 then term(a)
  else
    let m = (a + b) / 2 in
    let left = bsplit(a, m) in

```

```

    let right = bsplit(m, b) in
    rat_add(left, right)

fn chudnovsky_pi(n_digits) =
  let n_terms = n_digits / 14 + 2 in
  let prec    = n_digits + 20 in
  let _bump   = set_precision(prec) in
  let s_rat   = bsplit(0, n_terms) in
  let s_real  = real_from_rat(s_rat, prec) in
  let c       = real_mul(real_from_int(426880),
                        real_sqrt(real_from_int(10005), prec)) in
  real_div(c, s_real)

let pi = chudnovsky_pi(10000) in
emit(real_to_string(pi, 10000))

```

v1 の PhiLang のように `compute = chudnovsky_binary_splitting` で逃げず、

- ・ 二分木再帰 (bsplit)
- ・ Chudnovsky 級数の k 番目項 (term)
- ・ 最終的な π 復元 ($426880 \cdot \sqrt{10005} / S$)

をすべて **PsiLang** のソースコード自身が書いている。実行は tree-walker interpreter (psilang.py) が Python の `bigint・Decimal` を上から借りるだけ。

検証結果 (2026-05-16)

検査項目	結果
出力サイズ	10,002 chars = "3." + 10,000 digits
先頭 102 桁の既知 π との一致	✓
Feynman point (999999 の最初の出現位置)	762 (well-known 値と一致)
1000 桁目	9 (OEIS A000796 と一致)
v1 PhiLang/chudnovsky_pi(10000) との bitwise 一致	✓
計算時間	3.03 秒 (Apple M2)
Python interpreter 行数	約 650 行 (R4 想定 1500 行を下回る)

3 手法の比較 (遅 / 中 / 速)

PsiLang のソースコードだけで π 10,000 桁を 3 通り (+1 補助) に計算した時間:

.psi ファイル	級数	内部表現	累積	実時間	比率
machin_rat.psi	Machin (1.4 桁/項)	Rat	線形	28.24 s	100×
chudnovsky_li	Chudnovsky (14 桁/項)	Rat	線形	3.49 s	12×
chudnovsky.psi	Chudnovsky	Rat	二分木	3.04 s	10×
machin.psi	Machin	Real (Decimal)	線形	0.30 s	1×

4 出力すべて **SHA256** が完全一致 (d44e2dba39a378…), つまり 10,000 桁すべて bit-wise に同じ値。詳細は親 ../REPORT.md §7.3 を参照。

主な知見: - 同じ **Machin** 公式を Rat で書くと 28 秒、Real で書くと 0.3 秒 — 94 倍の差。- 級数の収束速度より、**型の選択 (Rat vs Real)** が性能を支配することが PsiLang で可視化される。- いずれの .psi も 50 行以下で書ける (R1_AlgorithmExpressivity の閾値内)。

v1 PhiLang との対比

観点	v1 PhiLang	v2 PsiLang
役割	Phase orchestration (timeout/retry/checkpoint)	Algorithm description (recursion/pattern/arithmetic)
π 計算ソース	compute = chudnovsky_binary_splitting (1 行 dispatch)	chudnovsky.psi (50 行の真のアルゴリズム)
ホスト依存	Python kernel philang_stdlib.chudnovsky_p	Python は int, Decimal, gcd を提供するだけ。algorithm は PsiLang 内
パラダイム	YAML-ish DSL	ML 系関数型
評価モデル	4 ブロック (phase / every / after / guarantees)	tree-walker + 再帰 + パターンマッチ
進化計算の根拠	top 5 共通の 4 つ巴 (bounded burst × mandatory retry × …)	top 5 コンセンサス (functional × static_simple × bigint+rat × pattern_match_fold × ml_like)

限界

- ・ **Closures** はサポートしない: 局所 let で定義した関数は環境を捕捉しない (fn 宣言はトップレベルのみ)。Chudnovsky には不要なので問題なし。
- ・ **型検査** なし: 型注釈は構文上受け取るが実行時は無視 (static_simple を「型注釈は人間用ドキュメント」と解釈)。
- ・ **末尾呼び出し最適化** なし: 深い線形再帰 (例: factorial(4000) を PsiLang 内で純粋再帰で書く) は Python 側で stack overflow になる。回避策として int_fact を builtin に置いた。

- ・ **lambdas (匿名関数)** なし: `fn` は宣言文のみ。FFT/quicksort を書くなら追加が必要。
- ・ **モジュール / import** なし: 単一ファイルプログラムのみ。